

What is Train-Test-Split?

The Train-Test-Split is a technique used in machine learning to evaluate the performance of a model. It involves splitting the dataset into two subsets: one for training the model and the other for testing its performance.

Why Train-Test-Split?

When building a machine learning model, it's crucial to assess how well it generalizes to unseen data. The goal is to build a model that can make accurate predictions on new, unseen data. The Train-Test-Split technique helps us achieve this by simulating the real-world scenario of having separate training and testing data.

How does Train-Test-Split work?

Preparing the Data: First, you start with a dataset that contains features (*input variables or independent variables*) and corresponding labels (*output variable or target or dependent variable*). Ensure that the data is cleaned and preprocessed appropriately.

Splitting the Data: The dataset is divided into two subsets:

Training Set: This subset is used to train the machine learning model. It typically contains around 70-80% of the total data.

Test Set: This subset is used to evaluate the performance of the trained model. It usually contains around 20-30% of the total data.

Training the Model: You train your machine learning model using the training set. The model learns patterns and relationships within the training data.

Testing the Model: Once the model is trained, you evaluate its performance using the test set. The model makes predictions on the test set, and these predictions are compared against the actual labels in the test set.

Performance Evaluation: Various metrics can be used to evaluate the model's performance on the test set, depending on the type of problem (e.g., classification, regression). Common metrics

include accuracy, precision, recall, F1-score (for classification), mean squared error, and R-squared (for regression).

Iterative Process: If the model's performance is unsatisfactory, you might need to adjust the model hyperparameters, feature engineering, or even try different algorithms and repeat the train-test split process until satisfactory performance is achieved.

Final Evaluation: Once you're satisfied with the model's performance on the test set, you can deploy it to make predictions on new, unseen data.

Key Considerations:

Randomness: The data should be randomly split into training and test sets to ensure that both sets are representative of the overall dataset.

Maintaining Data Distribution: Ensure that both the training and test sets maintain the same distribution of classes or labels to avoid biased evaluation.

Cross-Validation: Train-Test-Split is a single split, but for more reliable performance estimation, techniques like k-fold cross-validation can be used.

Example 01:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample dataset (features and target)
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

# Split the dataset into training and test sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Initialize and train the model
model = LinearRegression()
model.fit(X_train, y_train)

# Evaluate the model on the test set
test_predictions = model.predict(X_test)

# Print the test predictions and actual values
```

```
print("Test Predictions:", test_predictions)
print("Actual Values:", y_test)
```

Explanation:

- We have a simple dataset with one feature and one target variable.
- We split the dataset into training and test sets using `train_test_split`.
- We train a linear regression model on the training set.
- We make predictions on the test set and compare them with the actual values.

This example illustrates how Train-Test-Split is used to train and evaluate a machine learning model using a simple dataset.

Make sure you have scikit-learn installed (*pip install scikit-learn*) to run all examples.

Example 02:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load dataset (example with Iris dataset containing 150 samples)
iris = load_iris()

# Features (sepal length, sepal width, petal length, petal width)
X = iris.data

# Target labels (species: 0 for Setosa, 1 for Versicolor, 2 for Virginica)
y = iris.target

# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Initialize Logistic Regression model
model = LogisticRegression()

# Train the model on the training data
model.fit(X_train, y_train)

# Predict the labels for the test set
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Explanation:

- Loads the Iris dataset.
- Splits the dataset into training and testing sets using `train_test_split`, where 80% of the data is used for training and 20% for testing.
- Initializes a Logistic Regression model.
- Trains the model on the training data.
- Predicts the labels for the test set.
- Computes the accuracy of the model by comparing the predicted labels with the actual labels from the test set.

Also, note that you can replace the Iris dataset with your own dataset by loading it using appropriate functions.

Example 03:

```
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load the Boston Housing dataset consists of 506 samples of houses
boston = load_boston()
X = boston.data
y = boston.target

# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Initialize Linear Regression model
model = LinearRegression()

# Train the model on the training data
model.fit(X_train, y_train)

# Predict the target values for the test set
y_pred = model.predict(X_test)

# Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R-squared:", r2)
```

Explanation:

- Loads the Boston Housing dataset.
- Splits the dataset into training and testing sets using `train_test_split`, with 80% of the data used for training and 20% for testing.
- Initializes a Linear Regression model.
- Trains the model on the training data.
- Predicts the target values for the test set.
- Evaluates the model's performance using Mean Squared Error (MSE) and R-squared metrics.

The Boston Housing dataset is a common benchmark dataset for regression tasks, containing features related to housing in Boston and target variable being the median value of owner-occupied homes.

Example 04:

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Generate some sample data
np.random.seed(42)
X = np.random.rand(100, 1) * 10 # Generate 100 random values between 0 and 10
y = 3 * X + 2 + np.random.randn(100, 1) # Linear relationship with some random noise

# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize Linear Regression model
model = LinearRegression()

# Train the model on the training data
model.fit(X_train, y_train)

# Predict the target values for the test set
y_pred = model.predict(X_test)

# Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R-squared:", r2)
```

Explanation:

- Generates some sample data with a linear relationship between X and y, adding some random noise.
- Splits the dataset into training and testing sets using `train_test_split`, with 80% of the data used for training and 20% for testing.
- Initializes a Linear Regression model.
- Trains the model on the training data.
- Predicts the target values for the test set.
- Evaluates the model's performance using Mean Squared Error (MSE) and R-squared metrics.

Make sure you have scikit-learn installed (`pip install scikit-learn`) to run this code. You can replace the sample data generation with your own dataset loading process.

What is Sklearn?

Scikit-learn, often abbreviated as sklearn, is a popular open-source machine learning library for the Python programming language. It provides simple and efficient tools for data mining and data analysis and is built on top of other scientific computing libraries such as NumPy, SciPy, and matplotlib.

Sklearn offers a wide range of machine learning algorithms for various tasks such as classification, regression, clustering, dimensionality reduction, and model selection. Additionally, it provides utilities for data preprocessing, model evaluation, and model tuning.

Key features of scikit-learn include:

Consistent Interface: Sklearn provides a consistent interface across different algorithms, making it easy to experiment with different models without needing to learn new syntax for each one.

Wide Range of Algorithms: Sklearn includes implementations of many machine learning algorithms, including linear models, support vector machines, decision trees, random forests, gradient boosting, k-nearest neighbors, clustering algorithms, etc.

Integration with Other Libraries: Sklearn integrates well with other Python libraries commonly used in data science, such as NumPy (for numerical computing), SciPy (for scientific computing), pandas (for data manipulation), and matplotlib (for data visualization).

Ease of Use: Sklearn is designed with simplicity and usability in mind, making it accessible to both beginners and experienced machine learning practitioners.

Documentation and Community Support: Sklearn provides comprehensive documentation, including tutorials and examples, which makes it easy to get started with machine learning tasks. Additionally, it has a large and active community of users and contributors who provide support and contribute to its development.

Overall, scikit-learn is a powerful and versatile library that forms the backbone of many machine learning projects in Python. Whether you're a beginner or an experienced practitioner, it offers a wide range of tools and resources to help you build and deploy machine learning models effectively.

Cross Validation

Cross-validation is a statistical technique used in machine learning and data analysis to evaluate how well a predictive model performs on unseen data. The main idea behind cross-validation is to partition the available data into subsets, train the model on a portion of the data (training set), and then evaluate its performance on the remaining portion (test set).

Here's how cross-validation generally works:

1. **Data Splitting:** The dataset is divided into two or more subsets. The most common method is k-fold cross-validation, where the data is divided into k subsets (or folds) of approximately equal size.
2. **Model Training and Validation:** The model is trained on k-1 subsets (training set) and evaluated on the remaining subset (validation set or test set). This process is repeated k times, with each of the k subsets used exactly once as the validation data.
3. **Performance Evaluation:** After training and evaluating the model k times, the performance metrics (such as accuracy, precision, recall, F1 score, etc.) are averaged across all the folds to obtain an overall estimation of the model's performance.

Cross-validation helps to assess how well a model generalizes to new data by simulating the process of training and testing on different subsets of data. It provides a more reliable estimate of model performance compared to simply splitting the data into a single training set and a single test set because it reduces the variance in performance estimation caused by the randomness of the data split.

Common variations of cross-validation include:

1. **k-Fold Cross-Validation:** The dataset is divided into k subsets, and the cross-validation process is repeated k times, with each subset used once as the validation data.
2. **Leave-One-Out Cross-Validation (LOOCV):** Each data point is used as the validation set once while the rest of the data points form the training set. This process is repeated for each data point, resulting in n iterations for a dataset with n data points.
3. **Stratified Cross-Validation:** Ensures that each fold of the cross-validation dataset has the same proportion of class labels as the original dataset. This is particularly useful for datasets with imbalanced class distributions.

Cross-validation helps in selecting models, tuning hyperparameters, and assessing the generalization ability of a model before deploying it to unseen data. It's an essential tool in the machine learning pipeline for ensuring robust and reliable model performance estimation.

K-Fold Cross Validation

K-Fold Cross-Validation is a widely used technique in machine learning for model evaluation, particularly in situations where there's a limited amount of data available. It's a resampling method that helps in assessing how well a predictive model generalizes to an independent dataset. Here's how K-Fold Cross-Validation works:

1. **Data Splitting:** The dataset is divided into k subsets or folds of approximately equal size. Each fold contains a portion of the original dataset.
2. **Training and Validation:** The model is trained and evaluated k times. In each iteration, one of the k folds is held out as the validation set, while the remaining $k-1$ folds are used as the training set. So, for each iteration, you have a different validation set and a different training set.
3. **Model Evaluation:** After training the model on $k-1$ folds and validating it on the held-out fold, performance metrics (e.g., accuracy, precision, recall, F1-score) are computed for that iteration. This process is repeated k times, each time using a different fold as the validation set.
4. **Performance Estimation:** The performance metrics obtained from each iteration are averaged to get an overall performance estimation of the model. This average performance across all k folds gives a more reliable indication of how well the model is expected to perform on unseen data compared to training on a single train-test split.

5. **Parameter Tuning:** K-Fold Cross-Validation is also used for hyperparameter tuning. Multiple sets of hyperparameters are evaluated using cross-validation, and the set that yields the best average performance across all folds is selected.

Example 01:

```
from sklearn.model_selection import KFold
import numpy as np

# Create some example data
X = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])
y = np.array([1, 2, 3, 4])

# Initialize KFold with number of splits
kf = KFold(n_splits=2)

# Perform cross-validation
for train_index, test_index in kf.split(X):
    print("Train Index:", train_index, ", Test Index:", test_index)
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    print("X_train:", X_train, ", y_train:", y_train)
    print("X_test:", X_test, ", y_test:", y_test)
    print()
```

Explanation:

- KFold is a cross-validation technique provided by scikit-learn, used for splitting the dataset into K consecutive folds (without shuffling by default).
- In this example, we create a simple dataset X and y.
- We initialize KFold with the number of splits, here n_splits=2, meaning the dataset will be divided into 2 folds.
- We iterate over the splits generated by KFold.split(X) using a for loop.
- In each iteration, KFold.split(X) returns the indices of the training and testing data for that split.
- We then use these indices to split the dataset into training and testing sets (X_train, X_test, y_train, y_test), and print them out.

This code demonstrates how to use KFold for splitting a dataset into training and testing sets for cross-validation purposes. It's a crucial technique in machine learning to assess the performance of models and to ensure generalization to unseen data.

Example 02:

```
from sklearn.model_selection import KFold
import pandas as pd
```

```
#Create dictionary of PAI result, based on PF and Attendance
dic = {'PF':      [90,50,40,60,30,70,30,20,80],
       'Att':     [60,50,70,50,30,40,60,35,20],
       'Result': [1,1,1,1,0,0,0,0,0]}

#Convert to Dataframe
dataset = pd.DataFrame(dic)

#Separate Features and Target columns
X = dataset.iloc[:, :2]
y = dataset['Result']

# Initialize KFold with number of splits
kf = KFold(n_splits=3)

foldNumber = 1
# Perform cross-validation
for train_index, test_index in kf.split(X):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]
    print("=====")
    print("Fold:", foldNumber)
    print("=====")
    print("X_Train:\n", X_train)
    print("Y_Train:\n", y_train)
    print()
    print("X_Test\n", X_test)
    print("Y_Test\n", y_test)
    print()
    foldNumber += 1
```

Advantages of K-Fold Cross-Validation:

- **Better Performance Estimate:** By averaging the performance over k folds, K-Fold Cross-Validation provides a more accurate estimate of model performance compared to a single train-test split.
- **Data Utilization:** It makes efficient use of available data, as each data point is used for both training and validation exactly once.
- **Robustness:** It reduces the impact of variability in the training-test split on the model evaluation, leading to more robust model assessment.

Variations of K-Fold Cross-Validation:

- **Stratified K-Fold Cross-Validation:** Ensures that each fold preserves the same proportion of class labels as the original dataset. It's particularly useful for classification tasks with imbalanced class distributions.
- **Repeated K-Fold Cross-Validation:** The process of K-Fold Cross-Validation is repeated multiple times with different random splits of the data. This helps in obtaining a more stable estimate of model performance.

In summary, K-Fold Cross-Validation is a valuable technique for assessing and optimizing model performance in machine learning, providing a robust and reliable estimate of how well the model will generalize to unseen data.

Example 01: k-fold cross-validation on the Boston Housing dataset using Linear Regression as the model.

```
from sklearn.datasets import load_boston
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression

# Load the Boston Housing dataset
boston = load_boston()
X = boston.data
y = boston.target

# Initialize Linear Regression model
model = LinearRegression()

# Perform k-fold cross-validation (let's use k=5 for this example)
k = 5
scores = cross_val_score(model, X, y, cv=k,
                          scoring='neg_mean_squared_error')

# Convert the negative MSE scores to positive
mse_scores = -scores

# Print the MSE scores for each fold
print("MSE Scores for each fold:", mse_scores)

# Calculate the mean and standard deviation of the MSE scores
mean_mse = mse_scores.mean()
std_mse = mse_scores.std()

print("\nMean MSE:", mean_mse)
print("Standard Deviation of MSE:", std_mse)
```

Explanation:

- Loads the Boston Housing dataset.

- Initializes a Linear Regression model.
- Performs k-fold cross-validation with k=5 using cross_val_score, specifying 'neg_mean_squared_error' as the scoring metric (since scikit-learn returns negative values for MSE).
- Prints the MSE scores for each fold.
- Calculates the mean and standard deviation of the MSE scores.

This example demonstrates how k-fold cross-validation can be used to evaluate the performance of a regression model on the Boston Housing dataset. You can adjust the value of k for a different number of folds.

Example 02: k-fold cross-validation on a binary classification task using the Breast Cancer Wisconsin dataset with Logistic Regression.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

# Load the Breast Cancer Wisconsin dataset
breast_cancer = load_breast_cancer()
X = breast_cancer.data
y = breast_cancer.target

# Initialize Logistic Regression model
model = LogisticRegression()

# Perform k-fold cross-validation (let's use k=5 for this example)
k = 5
scores = cross_val_score(model, X, y, cv=k, scoring='accuracy')

# Print the accuracy scores for each fold
print("Accuracy Scores for each fold:", scores)

# Calculate the mean and standard deviation of the accuracy scores
mean_accuracy = scores.mean()
std_accuracy = scores.std()

print("\nMean Accuracy:", mean_accuracy)
print("Standard Deviation of Accuracy:", std_accuracy)
```

Explanation:

- Loads the Breast Cancer Wisconsin dataset, which is a binary classification task.
- Initializes a Logistic Regression model.
- Performs k-fold cross-validation with k=5 using cross_val_score, specifying 'accuracy' as the scoring metric.

- Prints the accuracy scores for each fold.
- Calculates the mean and standard deviation of the accuracy scores.

This example demonstrates how k-fold cross-validation can be used to evaluate the performance of a logistic regression model on the Breast Cancer Wisconsin dataset. You can adjust the value of k for a different number of folds.

Example 03: k-fold cross-validation on the Iris dataset using Logistic Regression for multi-class classification.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Initialize Logistic Regression model
model = LogisticRegression(max_iter=1000) # Increase max_iter to ensure
convergence

# Perform k-fold cross-validation (let's use k=5 for this example)
k = 5
scores = cross_val_score(model, X, y, cv=k, scoring='accuracy')

# Print the accuracy scores for each fold
print("Accuracy Scores for each fold:", scores)

# Calculate the mean and standard deviation of the accuracy scores
mean_accuracy = scores.mean()
std_accuracy = scores.std()

print("\nMean Accuracy:", mean_accuracy)
print("Standard Deviation of Accuracy:", std_accuracy)
```

Explanation:

- Loads the Iris dataset, which is a multi-class classification task.
- Initializes a Logistic Regression model.
- Performs k-fold cross-validation with k=5 using cross_val_score, specifying 'accuracy' as the scoring metric.
- Prints the accuracy scores for each fold.
- Calculates the mean and standard deviation of the accuracy scores.

This example demonstrates how k-fold cross-validation can be used to evaluate the performance of a logistic regression model on the Iris dataset. You can adjust the value of k for a different number of folds.

Classification vs Regression

Classification and regression are two main types of problems in machine learning, and they differ primarily in the nature of the output variable they aim to predict.

Classification:

In classification problems, the goal is to predict the categorical class labels of new instances based on past observations. The output variable is discrete and represents different categories or classes. Classification tasks include:

- **Binary Classification:** The output variable has only two possible classes (e.g., spam detection, fraud detection).
- **Multi-class Classification:** The output variable has more than two possible classes (e.g., image classification, sentiment analysis).
- **Multi-label Classification:** Each instance can belong to multiple classes simultaneously (e.g., document categorization).

In classification, the output is a discrete value representing a category, and algorithms are trained to assign input data points to one of these predefined categories based on features extracted from the data.

Regression:

In regression problems, the goal is to predict a continuous numeric value rather than a categorical label. The output variable is continuous, and the prediction is made based on the relationship between input variables (features) and the target variable. Regression tasks include:

- **Predicting Housing Prices:** Given features like size, location, number of bedrooms, etc., predict the price of a house.
- **Forecasting Stock Prices:** Given historical data and other relevant factors, predict the future price of a stock.
- **Estimating Sales Revenue:** Predict the sales revenue of a product based on factors like advertising expenditure, seasonality, etc.

In regression, the output is a real-valued number, and algorithms are trained to learn the relationship between input variables and the continuous target variable.

Key Differences:

- **Output Type:** Classification predicts categorical labels, while regression predicts continuous numeric values.
- **Evaluation Metrics:** Different metrics are used to evaluate the performance of classification and regression models. For classification, metrics like accuracy, precision, recall, F1-score are common, while regression models are evaluated using metrics such as mean squared error (MSE), mean absolute error (MAE), R-squared, etc.
- **Algorithms:** While some algorithms can be applied to both classification and regression problems (e.g., decision trees, neural networks), there are also algorithms specifically designed for each type of problem (e.g., logistic regression for classification, linear regression for regression).

In summary, the choice between classification and regression depends on the nature of the problem and the type of output variable you want to predict.

Different Evaluation Metrics

Evaluation metrics are quantitative measures used to assess the performance of machine learning models. These metrics provide insights into how well a model is performing in terms of its ability to make accurate predictions. Different metrics are used for different types of tasks, such as classification or regression. Common evaluation metrics include accuracy, precision, recall, F1-score for classification tasks, and mean squared error (MSE) and R-squared for regression tasks. Here is a brief explanation for each of these evaluation metrics:

Accuracy:

Accuracy measures the proportion of correct predictions out of the total number of predictions made by the model. It's a simple and intuitive metric but may not be suitable for imbalanced datasets where the classes are unevenly distributed.

Precision:

Precision measures the proportion of true positive predictions out of all positive predictions made by the model. It focuses on the accuracy of positive predictions and is useful when the cost of false positives is high.

Recall:

Recall, also known as sensitivity or true positive rate, measures the proportion of true positives that are correctly identified by the model out of all actual positives. It's important when the cost of false negatives is high, as it indicates the model's ability to capture all positive instances.

F1-score:

F1-score is the harmonic mean of precision and recall. It provides a balance between precision and recall and is useful when you need to consider both false positives and false negatives. F1-score reaches its best value at 1 (perfect precision and recall) and worst at 0.

Mean Squared Error (MSE):

MSE measures the average squared difference between the actual values and the predicted values. It's commonly used in regression tasks to quantify the model's accuracy by penalizing large errors more heavily. A lower MSE indicates better model performance.

R-squared (R²):

R-squared measures the proportion of the variance in the dependent variable that is predictable from the independent variables. It's a statistical measure of how well the regression line fits the data. R² ranges from 0 to 1, with 1 indicating a perfect fit and 0 indicating that the model does not explain any variability in the target variable beyond the mean.

Stratified K Fold Cross Validation

Stratified K Fold Cross-validation is a variant of the K Fold technique commonly employed in machine learning for evaluating model performance. Unlike traditional K Fold, where data is randomly partitioned into K equally sized folds, Stratified K Fold aims to maintain the class distribution within each fold, particularly crucial for classification tasks. This technique ensures that each fold retains a proportional representation of different classes, reducing the risk of bias and yielding more robust performance estimates.

In Stratified K Fold, the dataset is divided into K folds, with each fold containing approximately the same percentage of samples from each class as the original dataset. This is achieved by preserving the class distribution while partitioning the data. Subsequently, models are trained and evaluated K times, with each fold serving as a validation set while the remaining folds are used for training. By averaging the performance metrics across all folds, Stratified K Fold provides a more

accurate estimate of the model's generalization ability, particularly when dealing with imbalanced datasets where certain classes may be underrepresented. Overall, Stratified K Fold Cross-validation stands as a valuable tool in model evaluation, offering improved reliability and mitigating potential biases in performance estimation, especially in classification scenarios.

Example 01:

```
from sklearn.model_selection import StratifiedKFold
import numpy as np

# Create some example data
X = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])
y = np.array([0, 0, 1, 1]) # Binary classification labels

# Initialize StratifiedKFold with number of splits
skf = StratifiedKFold(n_splits=2)

# Perform cross-validation
for train_index, test_index in skf.split(X, y):
    print("Train Index:", train_index, ", Test Index:", test_index)
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    print("X_train:", X_train, ", y_train:", y_train)
    print("X_test:", X_test, ", y_test:", y_test)
    print()
```

Explanation:

- StratifiedKFold is a variation of KFold that provides stratified folds, preserving the percentage of samples for each class.
- In this example, we create a simple dataset X and y for binary classification.
- We initialize StratifiedKFold with the number of splits, here n_splits=2.
- We iterate over the splits generated by StratifiedKFold.split(X, y) using a for loop.
- In each iteration, StratifiedKFold.split(X, y) returns the indices of the training and testing data for that split while preserving the class distribution.
- We then use these indices to split the dataset into training and testing sets (X_train, X_test, y_train, y_test), and print them out.

This code demonstrates how to use StratifiedKFold for splitting a dataset into training and testing sets while preserving the class distribution, which is particularly useful for classification tasks. It helps ensure that each fold has a representative distribution of classes, making the cross-validation results more reliable.

Example 02:

```
from sklearn.model_selection import StratifiedKFold
import pandas as pd

#Create dictionary of PAI result, based on PF and Attendance
dic = {'PF':      [90,50,40,60,30,70,30,20,80],
       'Att':     [60,50,70,50,30,40,60,35,20],
       'Result': [1,1,1,1,0,0,0,0,0]}

#Convert to Dataframe
dataset = pd.DataFrame(dic)

#Separate Features and Target columns
X = dataset.iloc[:, :2]
y = dataset['Result']

# Initialize StratifiedKFold with number of splits
skf = StratifiedKFold(n_splits=3)

foldNumber = 1
# Perform cross-validation
for train_index, test_index in skf.split(X, y):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]
    print("=====")
    print("Fold:", foldNumber)
    print("=====")
    print("X_Train:\n", X_train)
    print("Y_Train:\n", y_train)
    print()
    print("X_Test\n", X_test)
    print("Y_Test\n", y_test)
    print()
    foldNumber += 1
```